

API Comandas — Documentação de Referência

Base URL: `https://ypvcxgkzolzxggfrmzlz.supabase.co/functions/v1/api-comandas`

Autenticação: Todas as requisições exigem o header `Authorization: Bearer <JWT>` obtido via `auth-login`.

Padrão arquitetural: Action-Based POST. Operações de leitura usam GET com query params; operações de escrita usam POST com `{ "action": "<nome>", ...payload }`.

1. Controle de Acesso

A função aplica três camadas de validação antes de processar qualquer request:

1. **JWT válido** — token verificado via `auth.getClaims`. Resposta 401 em caso de falha.
2. **Permissão de módulo** (sub-usuários) — exige `restaurant` **ou** `comandas` habilitado em `user_module_permissions`. Owners (proprietários) têm acesso total. Resposta 403 quando negado.
3. **Modo de operação** — se `operation_modes.comanda_enabled = false` para o owner, retorna 403 (Comanda mode is disabled for this company). Quando o registro não existe, o acesso é permitido (default ligado).

`ownerId` é resolvido automaticamente: para sub-usuários é o `owner_user_id` do registro em `company_users`; para owners é o próprio `auth.uid()`. Todo dado é segregado por `ownerId`.

2. Endpoints GET

2.1 Listar comandas (com filtros)

```
GET /api-comandas
  ?status=ABERTA
  &mesa_id=<uuid|none>
  &data_inicio=<ISO>
  &data_fim=<ISO>
  &q=<texto>
  &limit=200
```

Query params (todos opcionais):

Param	Tipo	Descrição
<code>status</code>	enum	ABERTA, EM_CONSUMO, AGUARDANDO_PAGAMENTO, FECHADA, CANCELADA. Sem este filtro, retorna apenas as abertas (3 primeiros).

mesa_id	uuid none	Filtra por mesa específica. Use none para comandas sem mesa (venda direta/balcão).
data_inicio	ISO 8601	Filtra data_abertura >= data_inicio.
data_fim	ISO 8601	Filtra data_abertura <= data_fim.
q	string	Busca textual ILIKE no campo observacao.
limit	int	Máx. 500. Default 200.

Resposta 200:

```
{
  "total": 12,
  "overview": {
    "total": 12,
    "abertas": 5,
    "aguardando_pagamento": 2,
    "fechadas": 4,
    "canceladas": 1,
    "total_liquido": 1842.50
  },
  "comandas": [
    {
      "id": "uuid",
      "mesa_id": "uuid|null",
      "status": "EM_CONSUMO",
      "data_abertura": "2026-04-26T17:30:00Z",
      "data_fechamento": null,
      "garcom_id": "string",
      "atendente_abertura_id": "string",
      "pessoas_qtd": 4,
      "observacao": "",
      "total_bruto": 120.0,
      "total_descontos": 0,
      "total_taxas": 0,
      "total_liquido": 120.0,
      "versao": 1,
      "sifen_cdc": null,
      "mesas": { "numero": 5, "setor": "Salão" }
    }
  ]
}
```

2.2 Detalhe de uma comanda

GET /api-comandas?id=<uuid>

Retorna a comanda + itens + pagamentos + bloco totais:

```
{
  "id": "uuid",
  "status": "AGUARDANDO_PAGAMENTO",
```

```

    "...": "todos os campos da comanda",
    "mesas": { "numero": 5, "setor": "Salão" },
    "itens": [ { "id": "uuid", "nome_snapshot": "X-Burger", "qtd": 2, "su
    "pagamentos": [ { "id": "uuid", "forma": "PIX", "valor": 50.0, "moeda
    "totais": {
      "total_pago": 50.0,
      "saldo": 70.0,
      "qtd_itens": 2
    }
  }
}

```

2.3 Recibo (payload consolidado para impressão)

GET /api-comandas?recibo=<uuid>

Retorna o pacote completo pronto para gerar recibo no PDV (impressora térmica 80mm, PDF, etc.):

```

{
  "comanda": { "...": "comanda completa com mesas{}" },
  "itens": [ "todos os itens (inclusive cancelados)" ],
  "itens_ativos": [ "apenas itens com status != CANCELADO" ],
  "pagamentos": [ "todos os pagamentos" ],
  "empresa": {
    "trade_name": "Plug PDV",
    "legal_name": "...",
    "document_number": "00.000.000/0001-00",
    "ruc": "",
    "phone": "",
    "email": "",
    "address": "Rua X",
    "address_number": "100",
    "neighborhood": "Centro",
    "city": "São Paulo",
    "state": "SP",
    "country": "BR",
    "preferred_currency": "BRL"
  },
  "totais": {
    "total_bruto": 120.0,
    "total_descontos": 0,
    "total_taxas": 0,
    "total_liquido": 120.0,
    "total_pago": 120.0,
    "saldo": 0.0,
    "qtd_itens": 2
  }
}

```

3. Endpoints POST (Actions)

Todos os POST aceitam JSON com a chave `action`. Erros comuns: 400 (validação), 403 (permissão/modo), 404 (não encontrado), 500 (erro interno).

3.1 abrir — Abrir comanda

```
POST /api-comandas
{
  "action": "abrir",
  "mesa_id": "uuid|null",
  "garcom_id": "string",
  "pessoas_qtd": 1,
  "observacao": ""
}
```

Cria a comanda com `status=ABERTA`. Se `mesa_id` for informada, marca a mesa como OCUPADA. Registra evento ABERTURA em `eventos_auditoria`.

Resposta: 201 com o objeto comanda.

3.2 add_item — Adicionar item

```
{
  "action": "add_item",
  "comanda_id": "uuid",
  "produto_id": "uuid",
  "qtd": 1,
  "observacao_item": "",
  "adicionais": {},
  "status": "ATIVO"
}
```

`status` é opcional (default ATIVO). Use RASCUNHO quando o item ainda está no carrinho do garçom e não deve ser enviado para a cozinha nem somar no total. Faz snapshot de `nome` e `selling_price` do produto, recalcula totais e registra evento ADD_ITEM.

Resposta: 201 com o item criado.

3.3 enviar_cozinha — Enviar itens em lote para cozinha

```
{
  "action": "enviar_cozinha",
  "comanda_id": "uuid",
  "item_ids": ["uuid", "uuid"]
}
```

Atualiza itens de RASCUNHO → ATIVO, marca `enviado_cozinha_em` com `timestamp`, recalcula totais e registra evento ENVIO_COZINHA. Se `item_ids` for omitido, envia **todos** os itens em rascunho da comanda.

Resposta: { `success`, `count`, `itens`: [...] }.

3.4 cancel_item — Cancelar item

```
{
  "action": "cancel_item",
  "comanda_id": "uuid",
  "item_id": "uuid",
  "motivo": "string"
}
```

Marca o item como CANCELADO, salva motivo_cancelamento, recalcula totais e registra REMOVE_ITEM.

3.5 update_item_status — Atualizar status do item (KDS)

```
{
  "action": "update_item_status",
  "comanda_id": "uuid",
  "item_id": "uuid",
  "status": "PREPARANDO"
}
```

Status válidos: RASCUNHO, ATIVO, PREPARANDO, PRONTO, ENTREGUE. Usado pelo Kitchen Display System (KDS).

3.6 add_pagamento — Registrar pagamento

```
{
  "action": "add_pagamento",
  "comanda_id": "uuid",
  "forma": "DINHEIRO",
  "valor": 50.0,
  "moeda": "BRL",
  "referencia_externa": ""
}
```

Formas: DINHEIRO, CREDITO, DEBITO, PIX, VOUCHER, OUTRO. Moedas suportadas: BRL, USD, EUR, PYG. Ao primeiro pagamento, a comanda passa para AGUARDANDO_PAGAMENTO. Registra evento PAGAMENTO.

3.7 fechar — Fechar comanda

```
{
  "action": "fechar",
  "comanda_id": "uuid"
}
```

Valida se total_pago >= total_liquido. Caso contrário, retorna 400 com { total_comanda, total_pago, faltando }. Em sucesso: marca FECHADA, libera a mesa (LIVRE), e — se sifen_config.enabled = true — dispara emissão automática de fatura eletrônica no Paraguai (não bloqueante).

Resposta: { success: true, comanda_id, total_pago, sifen_cdc }.

3.8 transferir_mesa — Transferir comanda entre mesas

```
{
  "action": "transferir_mesa",
  "comanda_id": "uuid",
  "mesa_destino_id": "uuid"
}
```

Valida que a mesa destino existe, pertence ao owner e está LIVRE. Atualiza a comanda, marca destino como OCUPADA e libera a mesa de origem. Registra TRANSFERENCIA.

3.9 cancelar — Cancelar comanda

```
{
  "action": "cancelar",
  "comanda_id": "uuid",
  "motivo": "string"
}
```

Marca a comanda como CANCELADA, libera mesa associada e registra CANCELAMENTO. Comandas já fechadas não podem ser canceladas.

4. Estados (Enums)

comanda_status: ABERTA, EM_CONSUMO, AGUARDANDO_PAGAMENTO, FECHADA, CANCELADA.

item_comanda_status: RASCUNHO, ATIVO, PREPARANDO, PRONTO, ENTREGUE, CANCELADO.

pagamento_status: REGISTRADO, CONFIRMADO, ESTORNADO.

forma_pagamento: DINHEIRO, CREDITO, DEBITO, PIX, VOUCHER, OUTRO.

mesa_status: LIVRE, OCUPADA, RESERVADA, MANUTENCAO.

5. Regras de Negócio Importantes

- **Recálculo de totais:** Ao adicionar/cancelar/enviar item, `recalcComanda` soma `subtotal` apenas dos itens com status diferente de CANCELADO e RASCUNHO. Itens em rascunho **não somam** no total.
- **Transição automática de status:** Quando há itens computáveis, a comanda vai para EM_CONSUMO; sem itens, volta para ABERTA. Após pagamento, vai para AGUARDANDO_PAGAMENTO.
- **Snapshot de preço:** O preço unitário é congelado no momento da inserção do item (`preco_unit_snapshot`), garantindo que mudanças no cadastro do produto não afetem comandas em andamento.
- **Auditoria obrigatória:** Toda operação crítica grava em `eventos_auditoria` com `actor_id` (quem executou) e `payload_json`.
- **Multi-moeda:** Pagamentos suportam BRL/USD/EUR/PYG. A conversão é responsabilidade do cliente (PDV) usando `exchange_rates`.
- **Sifen (Paraguai):** A emissão é não-bloqueante — falhas não impedem o fechamento da comanda.

6. Códigos HTTP

Código	Significado
200	OK (GET)
201	Created (POST de criação)
400	Erro de validação ou regra de negócio
401	JWT ausente, inválido ou expirado
403	Sem permissão de módulo ou modo desabilitado
404	Comanda/item/produto/mesa não encontrado
405	Método HTTP não suportado
500	Erro interno (ver logs da edge function)

7. Exemplos de Fluxo Completo

Fluxo Restaurante (mesa)

1. POST `abrir` com `mesa_id` → comanda criada, mesa OCUPADA.
2. POST `add_item` (`status=RASCUNHO`) repetido enquanto o garçom anota.
3. POST `enviar_cozinha` → itens viram ATIVO, KDS recebe.
4. POST `update_item_status` (KDS) → PREPARANDO → PRONTO → ENTREGUE.
5. POST `add_pagamento` → comanda AGUARDANDO_PAGAMENTO.
6. POST `fechar` → comanda FECHADA, mesa LIVRE, Sifen disparado.
7. GET `?recibo=<id>` → PDV imprime o recibo.

Fluxo Venda Direta / Balcão

1. POST `abrir` **sem** `mesa_id`.
2. POST `add_item` (`status=ATIVO` direto).
3. POST `add_pagamento` + POST `fechar` em sequência.

Fluxo Comanda Avulsa (sem mesa, com cartão físico)

Mesmo de venda direta, usando `observacao` para registrar o número do cartão/comanda física.

8. Headers Padrão

Authorization: Bearer <jwt>
Content-Type: application/json

CORS liberado para todas as origens.